

Numpy-Arrays

1) What is an Array?

⇒ An array is a data structure used to store multiple values in a single variable. In Numpy, an array is called an ndarray, which means N-dimensional array.

Eg: Import numpy as np

```
arr = np.array([10, 20, 30, 40, 50])
```

* It's uses & advantages:

- 1) It is generally designed specifically for numerical and mathematical operation.
- 2) Faster than python lists for many numerical operation.
- 3) Efficient memory usage for numerical data.
- 4) Supports mathematical operation.
- 5) Useful for data analysis, machine learning and scientific computing.

→ Eg:

```
arr = np.array([10, 20, 30])
```

```
print(arr * 2)
```

output: [20, 40, 60]



Differences between python list and Numpy Array

- 1D Array :

```
arr = np.array([10, 20, 30, 40])
```

- 2D Array : Contain row and column

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
```

- 3D Array : It can be thought of as multiple 2D arrays or layers.

```
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

* Shape of an Array

Example:

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
```

```
print(arr.shape)
```

Output: (2, 3)

↳ 2 rows X 3 columns

* Array Data Type

Example:

```
arr = np.array([1, 2, 3])
print(arr.dtype)
```

index allow int,
float, bool, complex
and str/obj types.

* Indexing of Arrays (Positive indexing)

Eg:

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])
print(arr[3])
```

output:

10
40

* Negative indexing

Eg:

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[-1])
```

output: 50

↓ ↓ ↓
-3 -2 -1
position

Array Slicing

Slicing means selecting a portion of an array.

Eg:

```
arr = np.array([10, 20, 30, 40, 50])
```

```
print(arr[1:4])
```

output: [20, 30, 40]

Syntax:

```
array[start:stop:step]
```

[Note: Stop index not included]

Mutable Arrays

Meaning: value can be changed after creation

Example:

```
arr = np.array([10, 20, 30])
```

```
arr[1] = 100
```

```
print(arr)
```

output: [10, 100, 30]

(*) Multidimensional Array (For 2d or 3d)

Example:

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
```

To access 50:

```
print(arr[1, 1])
```

↑ ↑
row column

(*) vector in Numpy (For one dimensional numpy array)

Example:

```
→ vector = np.array([10, 20, 30, 40])
```

A 10 element vector can be created as:

```
vector = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

- It's shape is (10).

(*) Matrix in Numpy (For 2D)

Example:

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
```

→ It's shape is (2, 3)

| |
row column
(2 x 3) matrix